laptop_price_prediction

August 8, 2026

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error
import seaborn as sns
from ISLP.models import (ModelSpec as MS, summarize, poly)
from statsmodels.stats.outliers_influence import variance_inflation_factor
import re
```

0.1 ETL Data Phase

[2]:
```
# Importing csv as data frame
laptop_origin = pd.read_csv("C:\\Users\\ihsan\\Data Analysis\\Laptop␣
 ↪Specifications and Price Prediction Dataset\\Data\\laptop_data.csv",
                    index_col=0)

laptop = laptop_origin.copy()

# Viewing dataframe sample
laptop.head()
```

[2]:

	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory
0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8GB	128GB SSD
1	Apple	Ultrabook	13.3	1440x900	Intel Core i5 1.8GHz	8GB	128GB Flash Storage
2	HP	Notebook	15.6	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8GB	256GB SSD
3	Apple	Ultrabook	15.4	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16GB	512GB SSD
4	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8GB	256GB SSD

```
                           Gpu  OpSys  Weight        Price
0  Intel Iris Plus Graphics 640  macOS  1.37kg   71378.6832
1        Intel HD Graphics 6000  macOS  1.34kg   47895.5232
2         Intel HD Graphics 620  No OS  1.86kg   30636.0000
3            AMD Radeon Pro 455  macOS  1.83kg  135195.3360
4  Intel Iris Plus Graphics 650  macOS  1.37kg   96095.8080
```

[3]:
```
# Checking memory usage
laptop.info(memory_usage='deep')
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1303 entries, 0 to 1302
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   Company           1303 non-null   object 
 1   TypeName          1303 non-null   object 
 2   Inches            1303 non-null   float64
 3   ScreenResolution  1303 non-null   object 
 4   Cpu               1303 non-null   object 
 5   Ram               1303 non-null   object 
 6   Memory            1303 non-null   object 
 7   Gpu               1303 non-null   object 
 8   OpSys             1303 non-null   object 
 9   Weight            1303 non-null   object 
 10  Price             1303 non-null   float64
dtypes: float64(2), object(9)
memory usage: 730.4 KB
```

[4]:
```
# Checking data type for each collumn
print(laptop.dtypes)

# Checking database size
print(laptop.shape)
```

```
Company              object
TypeName             object
Inches              float64
ScreenResolution     object
Cpu                  object
Ram                  object
Memory               object
Gpu                  object
OpSys                object
Weight               object
Price               float64
dtype: object
```

```
(1303, 11)
```

[5]:
```
# Data Cleaning, preparation for data analysis
# Checking all value of column 'Weight', if there is Kg on the ending of all␣
 ↪value remove it.
laptop['Weight'].unique()
```

[5]:
```
array(['1.37kg', '1.34kg', '1.86kg', '1.83kg', '2.1kg', '2.04kg', '1.3kg',
       '1.6kg', '2.2kg', '0.92kg', '1.22kg', '0.98kg', '2.5kg', '1.62kg',
       '1.91kg', '2.3kg', '1.35kg', '1.88kg', '1.89kg', '1.65kg',
       '2.71kg', '1.2kg', '1.44kg', '2.8kg', '2kg', '2.65kg', '2.77kg',
       '3.2kg', '0.69kg', '1.49kg', '2.4kg', '2.13kg', '2.43kg', '1.7kg',
       '1.4kg', '1.8kg', '1.9kg', '3kg', '1.252kg', '2.7kg', '2.02kg',
       '1.63kg', '1.96kg', '1.21kg', '2.45kg', '1.25kg', '1.5kg',
       '2.62kg', '1.38kg', '1.58kg', '1.85kg', '1.23kg', '1.26kg',
       '2.16kg', '2.36kg', '2.05kg', '1.32kg', '1.75kg', '0.97kg',
       '2.9kg', '2.56kg', '1.48kg', '1.74kg', '1.1kg', '1.56kg', '2.03kg',
       '1.05kg', '4.4kg', '1.90kg', '1.29kg', '2.0kg', '1.95kg', '2.06kg',
       '1.12kg', '1.42kg', '3.49kg', '3.35kg', '2.23kg', '4.42kg',
       '2.69kg', '2.37kg', '4.7kg', '3.6kg', '2.08kg', '4.3kg', '1.68kg',
       '1.41kg', '4.14kg', '2.18kg', '2.24kg', '2.67kg', '2.14kg',
       '1.36kg', '2.25kg', '2.15kg', '2.19kg', '2.54kg', '3.42kg',
       '1.28kg', '2.33kg', '1.45kg', '2.79kg', '1.84kg', '2.6kg',
       '2.26kg', '3.25kg', '1.59kg', '1.13kg', '1.78kg', '1.10kg',
       '1.15kg', '1.27kg', '1.43kg', '2.31kg', '1.16kg', '1.64kg',
       '2.17kg', '1.47kg', '3.78kg', '1.79kg', '0.91kg', '1.99kg',
       '4.33kg', '1.93kg', '1.87kg', '2.63kg', '3.4kg', '3.14kg',
       '1.94kg', '1.24kg', '4.6kg', '4.5kg', '2.73kg', '1.39kg', '2.29kg',
       '2.59kg', '2.94kg', '1.14kg', '3.8kg', '3.31kg', '1.09kg',
       '3.21kg', '1.19kg', '1.98kg', '1.17kg', '4.36kg', '1.71kg',
       '2.32kg', '4.2kg', '1.55kg', '0.81kg', '1.18kg', '2.72kg',
       '1.31kg', '0.920kg', '3.74kg', '1.76kg', '1.54kg', '2.83kg',
       '2.07kg', '2.38kg', '3.58kg', '1.08kg', '2.20kg', '2.75kg',
       '1.70kg', '2.99kg', '1.11kg', '2.09kg', '4kg', '3.0kg', '0.99kg',
       '3.52kg', '2.591kg', '2.21kg', '3.3kg', '2.191kg', '2.34kg',
       '4.0kg'], dtype=object)
```

[6]:
```
# Removing the last 2 digit from Weight collumn
laptop['Weight'] = laptop['Weight'].str[:-2]

# Changing dtype from object to float
laptop['Weight'] = laptop['Weight'].astype(float)
```

[7]:
```
# Checking to see if it has been fixed
laptop['Weight'].head()
```

[7]:
```
0    1.37
1    1.34
2    1.86
3    1.83
4    1.37
Name: Weight, dtype: float64
```

[8]:
```
# Checking all unique value of Ram Column, to decide how to remove the Gb
laptop['Ram'].unique()
```

[8]:
```
array(['8GB', '16GB', '4GB', '2GB', '12GB', '6GB', '32GB', '24GB', '64GB'],
      dtype=object)
```

[9]:
```
# Removing the last 2 digit from Ram collumn
laptop['Ram'] = laptop['Ram'].str[:-2]

# Changing dtype from object to interger
laptop['Ram'] = laptop['Ram'].astype(int)
```

[10]:
```
# Re-check
laptop['Ram'].head()
```

[10]:
```
0     8
1     8
2     8
3    16
4     8
Name: Ram, dtype: int64
```

[11]:
```
# Checking OpSys all unique value
laptop['OpSys'].unique()
```

[11]:
```
array(['macOS', 'No OS', 'Windows 10', 'Mac OS X', 'Linux', 'Android',
       'Windows 10 S', 'Chrome OS', 'Windows 7'], dtype=object)
```

[12]:
```
# Only need to change the dtype to category
laptop['OpSys'] = laptop['OpSys'].astype('category')
```

[13]:
```
# Re-check
laptop['OpSys'].head()
```

[13]:
```
0    macOS
1    macOS
2    No OS
3    macOS
4    macOS
Name: OpSys, dtype: category
Categories (9, object): ['Android', 'Chrome OS', 'Linux', 'Mac OS X', …,
```

```
'Windows 10', 'Windows 10 S', 'Windows 7', 'macOS']
```

[14]:
```
# Checking all unique value of memory column
laptop['Memory'].unique()
```

[14]:
```
array(['128GB SSD', '128GB Flash Storage', '256GB SSD', '512GB SSD',
       '500GB HDD', '256GB Flash Storage', '1TB HDD',
       '32GB Flash Storage', '128GB SSD +  1TB HDD',
       '256GB SSD +  256GB SSD', '64GB Flash Storage',
       '256GB SSD +  1TB HDD', '256GB SSD +  2TB HDD', '32GB SSD',
       '2TB HDD', '64GB SSD', '1.0TB Hybrid', '512GB SSD +  1TB HDD',
       '1TB SSD', '256GB SSD +  500GB HDD', '128GB SSD +  2TB HDD',
       '512GB SSD +  512GB SSD', '16GB SSD', '16GB Flash Storage',
       '512GB SSD +  256GB SSD', '512GB SSD +  2TB HDD',
       '64GB Flash Storage +  1TB HDD', '180GB SSD', '1TB HDD +  1TB HDD',
       '32GB HDD', '1TB SSD +  1TB HDD', '512GB Flash Storage',
       '128GB HDD', '240GB SSD', '8GB SSD', '508GB Hybrid', '1.0TB HDD',
       '512GB SSD +  1.0TB Hybrid', '256GB SSD +  1.0TB Hybrid'],
      dtype=object)
```

[15]:
```
# Extracting value of the Memory column into a more robust categorical column␣
 ↪for analysis

# Creating new column
laptop['Memory_SSD'] = 0
laptop['Memory_HDD'] = 0
laptop['Memory_Hybrid'] = 0
laptop['Memory_Flash'] = 0

for idx, mem in enumerate(laptop['Memory']): #enumerate to spearate the index &␣
 ↪value
    matches = re.findall(
        r'(\d+(?:\.\d+)?)\s*(TB|GB)\s*(SSD|HDD|Hybrid|Flash Storage)',
        mem
    ) #first category separating integer and float, second is TB and Gb, last␣
 ↪one separating drives type

    for size, unit, drive_type in matches:

        size = float(size)
        if unit == 'TB': #Coverting all TB Values into GB for data unity
            size *= 1024

        if drive_type == 'SSD':
            laptop.loc[idx, 'Memory_SSD'] += size

        elif drive_type == 'HDD':
```

```
            laptop.loc[idx, 'Memory_HDD'] += size

        elif drive_type == 'Hybrid':
            laptop.loc[idx, 'Memory_Hybrid'] += size

        elif drive_type == 'Flash Storage':
            laptop.loc[idx, 'Memory_Flash'] += size
```

[16]:
```
# Re-check
laptop.head()
```

[16]:
```
  Company   TypeName  Inches                    ScreenResolution  \
0   Apple  Ultrabook    13.3  IPS Panel Retina Display 2560x1600
1   Apple  Ultrabook    13.3                            1440x900
2      HP   Notebook    15.6                  Full HD 1920x1080
3   Apple  Ultrabook    15.4  IPS Panel Retina Display 2880x1800
4   Apple  Ultrabook    13.3  IPS Panel Retina Display 2560x1600

                          Cpu  Ram               Memory  \
0        Intel Core i5 2.3GHz    8            128GB SSD
1        Intel Core i5 1.8GHz    8  128GB Flash Storage
2  Intel Core i5 7200U 2.5GHz    8            256GB SSD
3        Intel Core i7 2.7GHz   16            512GB SSD
4        Intel Core i5 3.1GHz    8            256GB SSD

                            Gpu  OpSys  Weight        Price  Memory_SSD  \
0  Intel Iris Plus Graphics 640  macOS    1.37   71378.6832         128
1        Intel HD Graphics 6000  macOS    1.34   47895.5232           0
2         Intel HD Graphics 620  No OS    1.86   30636.0000         256
3            AMD Radeon Pro 455  macOS    1.83  135195.3360         512
4  Intel Iris Plus Graphics 650  macOS    1.37   96095.8080         256

   Memory_HDD  Memory_Hybrid  Memory_Flash
0           0              0             0
1           0              0           128
2           0              0             0
3           0              0             0
4           0              0             0
```

[17]:
```
# checking all unique value of GPU column
laptop['Gpu'].unique()
```

[17]:
```
array(['Intel Iris Plus Graphics 640', 'Intel HD Graphics 6000',
       'Intel HD Graphics 620', 'AMD Radeon Pro 455',
       'Intel Iris Plus Graphics 650', 'AMD Radeon R5',
       'Intel Iris Pro Graphics', 'Nvidia GeForce MX150',
       'Intel UHD Graphics 620', 'Intel HD Graphics 520',
```

```
 'AMD Radeon Pro 555', 'AMD Radeon R5 M430',
 'Intel HD Graphics 615', 'AMD Radeon Pro 560',
 'Nvidia GeForce 940MX', 'Intel HD Graphics 400',
 'Nvidia GeForce GTX 1050', 'AMD Radeon R2', 'AMD Radeon 530',
 'Nvidia GeForce 930MX', 'Intel HD Graphics',
 'Intel HD Graphics 500', 'Nvidia GeForce 930MX ',
 'Nvidia GeForce GTX 1060', 'Nvidia GeForce 150MX',
 'Intel Iris Graphics 540', 'AMD Radeon RX 580',
 'Nvidia GeForce 920MX', 'AMD Radeon R4 Graphics', 'AMD Radeon 520',
 'Nvidia GeForce GTX 1070', 'Nvidia GeForce GTX 1050 Ti',
 'Nvidia GeForce MX130', 'AMD R4 Graphics',
 'Nvidia GeForce GTX 940MX', 'AMD Radeon RX 560',
 'Nvidia GeForce 920M', 'AMD Radeon R7 M445', 'AMD Radeon RX 550',
 'Nvidia GeForce GTX 1050M', 'Intel HD Graphics 515',
 'AMD Radeon R5 M420', 'Intel HD Graphics 505',
 'Nvidia GTX 980 SLI', 'AMD R17M-M1-70', 'Nvidia GeForce GTX 1080',
 'Nvidia Quadro M1200', 'Nvidia GeForce 920MX ',
 'Nvidia GeForce GTX 950M', 'AMD FirePro W4190M ',
 'Nvidia GeForce GTX 980M', 'Intel Iris Graphics 550',
 'Nvidia GeForce 930M', 'Intel HD Graphics 630',
 'AMD Radeon R5 430', 'Nvidia GeForce GTX 940M',
 'Intel HD Graphics 510', 'Intel HD Graphics 405',
 'AMD Radeon RX 540', 'Nvidia GeForce GT 940MX',
 'AMD FirePro W5130M', 'Nvidia Quadro M2200M', 'AMD Radeon R4',
 'Nvidia Quadro M620', 'AMD Radeon R7 M460',
 'Intel HD Graphics 530', 'Nvidia GeForce GTX 965M',
 'Nvidia GeForce GTX1080', 'Nvidia GeForce GTX1050 Ti',
 'Nvidia GeForce GTX 960M', 'AMD Radeon R2 Graphics',
 'Nvidia Quadro M620M', 'Nvidia GeForce GTX 970M',
 'Nvidia GeForce GTX 960<U+039C>', 'Intel Graphics 620',
 'Nvidia GeForce GTX 960', 'AMD Radeon R5 520',
 'AMD Radeon R7 M440', 'AMD Radeon R7', 'Nvidia Quadro M520M',
 'Nvidia Quadro M2200', 'Nvidia Quadro M2000M',
 'Intel HD Graphics 540', 'Nvidia Quadro M1000M', 'AMD Radeon 540',
 'Nvidia GeForce GTX 1070M', 'Nvidia GeForce GTX1060',
 'Intel HD Graphics 5300', 'AMD Radeon R5 M420X',
 'AMD Radeon R7 Graphics', 'Nvidia GeForce 920',
 'Nvidia GeForce 940M', 'Nvidia GeForce GTX 930MX',
 'AMD Radeon R7 M465', 'AMD Radeon R3', 'Nvidia GeForce GTX 1050Ti',
 'AMD Radeon R7 M365X', 'AMD Radeon R9 M385',
 'Intel HD Graphics 620 ', 'Nvidia Quadro 3000M',
 'Nvidia GeForce GTX 980 ', 'AMD Radeon R5 M330',
 'AMD FirePro W4190M', 'AMD FirePro W6150M', 'AMD Radeon R5 M315',
 'Nvidia Quadro M500M', 'AMD Radeon R7 M360',
 'Nvidia Quadro M3000M', 'Nvidia GeForce 960M', 'ARM Mali T860 MP4'],
dtype=object)
```

[18]:
```
# Extracting value of the GPU column into a more robust categorical column for␣
↪analysis

# Creating new column to extract Gpu manufacturer
laptop['Gpu_Nvidia'] = laptop['Gpu'].str.contains('Nvidia').astype(int)
laptop['Gpu_AMD'] = laptop['Gpu'].str.contains('AMD').astype(int)
laptop['Gpu_Intel'] = laptop['Gpu'].str.contains('Intel').astype(int)
laptop['Gpu_ARM'] = laptop['Gpu'].str.contains('ARM').astype(int)
```

[19]:
```
# Re-check
laptop.head()
```

[19]:

	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price	Memory_SSD	Memory_HDD	Memory_Hybrid	Memory_Flash	Gpu_Nvidia	Gpu_AMD	Gpu_Intel	Gpu_ARM
0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	128	0	0	0	0	0	1	0
1	Apple	Ultrabook	13.3	1440x900	Intel Core i5 1.8GHz	8	128GB Flash Storage	Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	0	128	0	0	1	0
2	HP	Notebook	15.6	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8	256GB SSD	Intel HD Graphics 620	No OS	1.86	30636.0000	256	0	0	0	0	0	1	0
3	Apple	Ultrabook	15.4	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16	512GB SSD	AMD Radeon Pro 455	macOS	1.83	135195.3360	512	0	0	0	0	1	0	0
4	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	256	0	0	0	0	0	1	0

[20]:
```
# Checking all unique value of CPU column
laptop['Cpu'].unique()
```

[20]:
```
array(['Intel Core i5 2.3GHz', 'Intel Core i5 1.8GHz',
       'Intel Core i5 7200U 2.5GHz', 'Intel Core i7 2.7GHz',
       'Intel Core i5 3.1GHz', 'AMD A9-Series 9420 3GHz',
       'Intel Core i7 2.2GHz', 'Intel Core i7 8550U 1.8GHz',
       'Intel Core i5 8250U 1.6GHz', 'Intel Core i3 6006U 2GHz',
       'Intel Core i7 2.8GHz', 'Intel Core M m3 1.2GHz',
       'Intel Core i7 7500U 2.7GHz', 'Intel Core i7 2.9GHz',
       'Intel Core i3 7100U 2.4GHz', 'Intel Atom x5-Z8350 1.44GHz',
       'Intel Core i5 7300HQ 2.5GHz', 'AMD E-Series E2-9000e 1.5GHz',
       'Intel Core i5 1.6GHz', 'Intel Core i7 8650U 1.9GHz',
       'Intel Atom x5-Z8300 1.44GHz', 'AMD E-Series E2-6110 1.5GHz',
       'AMD A6-Series 9220 2.5GHz',
       'Intel Celeron Dual Core N3350 1.1GHz',
       'Intel Core i3 7130U 2.7GHz', 'Intel Core i7 7700HQ 2.8GHz',
       'Intel Core i5 2.0GHz', 'AMD Ryzen 1700 3GHz',
       'Intel Pentium Quad Core N4200 1.1GHz',
       'Intel Atom x5-Z8550 1.44GHz',
       'Intel Celeron Dual Core N3060 1.6GHz', 'Intel Core i5 1.3GHz',
       'AMD FX 9830P 3GHz', 'Intel Core i7 7560U 2.4GHz',
       'AMD E-Series 6110 1.5GHz', 'Intel Core i5 6200U 2.3GHz',
       'Intel Core M 6Y75 1.2GHz', 'Intel Core i5 7500U 2.7GHz',
       'Intel Core i3 6006U 2.2GHz', 'AMD A6-Series 9220 2.9GHz',
       'Intel Core i7 6920HQ 2.9GHz', 'Intel Core i5 7Y54 1.2GHz',
       'Intel Core i7 7820HK 2.9GHz', 'Intel Xeon E3-1505M V6 3GHz',
       'Intel Core i7 6500U 2.5GHz', 'AMD E-Series 9000e 1.5GHz',
       'AMD A10-Series A10-9620P 2.5GHz', 'AMD A6-Series A6-9220 2.5GHz',
       'Intel Core i5 2.9GHz', 'Intel Core i7 6600U 2.6GHz',
       'Intel Core i3 6006U 2.0GHz',
       'Intel Celeron Dual Core 3205U 1.5GHz',
       'Intel Core i7 7820HQ 2.9GHz', 'AMD A10-Series 9600P 2.4GHz',
       'Intel Core i7 7600U 2.8GHz', 'AMD A8-Series 7410 2.2GHz',
       'Intel Celeron Dual Core 3855U 1.6GHz',
       'Intel Pentium Quad Core N3710 1.6GHz',
       'AMD A12-Series 9720P 2.7GHz', 'Intel Core i5 7300U 2.6GHz',
       'AMD A12-Series 9720P 3.6GHz',
       'Intel Celeron Quad Core N3450 1.1GHz',
       'Intel Celeron Dual Core N3060 1.60GHz',
       'Intel Core i5 6440HQ 2.6GHz', 'Intel Core i7 6820HQ 2.7GHz',
       'AMD Ryzen 1600 3.2GHz', 'Intel Core i7 7Y75 1.3GHz',
       'Intel Core i5 7440HQ 2.8GHz', 'Intel Core i7 7660U 2.5GHz',
       'Intel Core i7 7700HQ 2.7GHz', 'Intel Core M m3-7Y30 2.2GHz',
       'Intel Core i5 7Y57 1.2GHz', 'Intel Core i7 6700HQ 2.6GHz',
       'Intel Core i3 6100U 2.3GHz', 'AMD A10-Series 9620P 2.5GHz',
       'AMD E-Series 7110 1.8GHz', 'Intel Celeron Dual Core N3350 2.0GHz',
```

```
       'AMD A9-Series A9-9420 3GHz', 'Intel Core i7 6820HK 2.7GHz',
       'Intel Core M 7Y30 1.0GHz', 'Intel Xeon E3-1535M v6 3.1GHz',
       'Intel Celeron Quad Core N3160 1.6GHz',
       'Intel Core i5 6300U 2.4GHz', 'Intel Core i3 6100U 2.1GHz',
       'AMD E-Series E2-9000 2.2GHz',
       'Intel Celeron Dual Core N3050 1.6GHz',
       'Intel Core M M3-6Y30 0.9GHz', 'AMD A9-Series 9420 2.9GHz',
       'Intel Core i5 6300HQ 2.3GHz', 'AMD A6-Series 7310 2GHz',
       'Intel Atom Z8350 1.92GHz', 'Intel Xeon E3-1535M v5 2.9GHz',
       'Intel Core i5 6260U 1.8GHz',
       'Intel Pentium Dual Core N4200 1.1GHz',
       'Intel Celeron Quad Core N3710 1.6GHz', 'Intel Core M 1.2GHz',
       'AMD A12-Series 9700P 2.5GHz', 'Intel Core i7 7500U 2.5GHz',
       'Intel Pentium Dual Core 4405U 2.1GHz',
       'AMD A4-Series 7210 2.2GHz', 'Intel Core i7 6560U 2.2GHz',
       'Intel Core M m7-6Y75 1.2GHz', 'AMD FX 8800P 2.1GHz',
       'Intel Core M M7-6Y75 1.2GHz', 'Intel Core i5 7200U 2.50GHz',
       'Intel Core i5 7200U 2.70GHz', 'Intel Atom X5-Z8350 1.44GHz',
       'Intel Core i5 7200U 2.7GHz', 'Intel Core M 1.1GHz',
       'Intel Pentium Dual Core 4405Y 1.5GHz',
       'Intel Pentium Quad Core N3700 1.6GHz', 'Intel Core M 6Y54 1.1GHz',
       'Intel Core i7 6500U 2.50GHz',
       'Intel Celeron Dual Core N3350 2GHz',
       'Samsung Cortex A72&A53 2.0GHz', 'AMD E-Series 9000 2.2GHz',
       'Intel Core M 6Y30 0.9GHz', 'AMD A9-Series 9410 2.9GHz'],
      dtype=object)
```

[21]:
```
# Extracting value of the CPU column into a more robust categorical column for analysis

laptop['Cpu_Intel'] = laptop['Cpu'].str.contains('Intel').astype(int)
laptop['Cpu_AMD'] = laptop['Cpu'].str.contains('AMD').astype(int)
laptop['Cpu_Samsung'] = laptop['Cpu'].str.contains('Samsung').astype(int)

laptop['Cpu_GHz'] = (
    laptop['Cpu']
    .str.extract(r'(\d+(?:\.\d+)?)GHz')
    .astype(float)
)
```

[22]:
```
# Re-check
laptop.head()
```

[22]:

	Company	TypeName	Inches	ScreenResolution	\
0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	
1	Apple	Ultrabook	13.3	1440x900	
2	HP	Notebook	15.6	Full HD 1920x1080	

```
3   Apple  Ultrabook    15.4  IPS Panel Retina Display 2880x1800
4   Apple  Ultrabook    13.3  IPS Panel Retina Display 2560x1600

                          Cpu  Ram               Memory  \
0        Intel Core i5 2.3GHz    8            128GB SSD
1        Intel Core i5 1.8GHz    8  128GB Flash Storage
2  Intel Core i5 7200U 2.5GHz    8            256GB SSD
3        Intel Core i7 2.7GHz   16            512GB SSD
4        Intel Core i5 3.1GHz    8            256GB SSD

                            Gpu  OpSys  Weight  …  Memory_Hybrid  \
0  Intel Iris Plus Graphics 640  macOS    1.37  …              0
1        Intel HD Graphics 6000  macOS    1.34  …              0
2         Intel HD Graphics 620  No OS    1.86  …              0
3            AMD Radeon Pro 455  macOS    1.83  …              0
4  Intel Iris Plus Graphics 650  macOS    1.37  …              0

   Memory_Flash  Gpu_Nvidia  Gpu_AMD  Gpu_Intel  Gpu_ARM  Cpu_Intel  Cpu_AMD  \
0             0           0        0          1        0          1        0
1           128           0        0          1        0          1        0
2             0           0        0          1        0          1        0
3             0           0        1          0        0          1        0
4             0           0        0          1        0          1        0

   Cpu_Samsung  Cpu_GHz
0            0      2.3
1            0      1.8
2            0      2.5
3            0      2.7
4            0      3.1

[5 rows x 23 columns]
```

[23]:
```
# Checking ScreenResolution all unique value
laptop['ScreenResolution'].unique()
```

[23]:
```
array(['IPS Panel Retina Display 2560x1600', '1440x900',
       'Full HD 1920x1080', 'IPS Panel Retina Display 2880x1800',
       '1366x768', 'IPS Panel Full HD 1920x1080',
       'IPS Panel Retina Display 2304x1440',
       'IPS Panel Full HD / Touchscreen 1920x1080',
       'Full HD / Touchscreen 1920x1080',
       'Touchscreen / Quad HD+ 3200x1800',
       'IPS Panel Touchscreen 1920x1200', 'Touchscreen 2256x1504',
       'Quad HD+ / Touchscreen 3200x1800', 'IPS Panel 1366x768',
       'IPS Panel 4K Ultra HD / Touchscreen 3840x2160',
       'IPS Panel Full HD 2160x1440',
```

```
       '4K Ultra HD / Touchscreen 3840x2160', 'Touchscreen 2560x1440',
       '1600x900', 'IPS Panel 4K Ultra HD 3840x2160',
       '4K Ultra HD 3840x2160', 'Touchscreen 1366x768',
       'IPS Panel Full HD 1366x768', 'IPS Panel 2560x1440',
       'IPS Panel Full HD 2560x1440',
       'IPS Panel Retina Display 2736x1824', 'Touchscreen 2400x1600',
       '2560x1440', 'IPS Panel Quad HD+ 2560x1440',
       'IPS Panel Quad HD+ 3200x1800',
       'IPS Panel Quad HD+ / Touchscreen 3200x1800',
       'IPS Panel Touchscreen 1366x768', '1920x1080',
       'IPS Panel Full HD 1920x1200',
       'IPS Panel Touchscreen / 4K Ultra HD 3840x2160',
       'IPS Panel Touchscreen 2560x1440',
       'Touchscreen / Full HD 1920x1080', 'Quad HD+ 3200x1800',
       'Touchscreen / 4K Ultra HD 3840x2160',
       'IPS Panel Touchscreen 2400x1600'], dtype=object)
```

[24]:
```
#Extracting resolution size from ScreenResolution
resolution = laptop['ScreenResolution'].str.extract(
    r'(\d+)x(\d+)'
)

# Inserting screen resolution into the laptop dataframe
laptop['Resolution_Width'] = resolution[0].astype(int)
laptop['Resolution_Height'] = resolution[1].astype(int)
#counting pixel density of the laptop screen
laptop['Pixels'] = (
    laptop['Resolution_Width']
    * laptop['Resolution_Height']
)
#categorizing the screen based on whether its using IPS panel or not
laptop['IPS'] = (
    laptop['ScreenResolution']
    .str.contains('IPS', na=False)
    .astype(int)
)
#categorizing the screen based on whether they are touchscreen or not
laptop['Touchscreen'] = (
    laptop['ScreenResolution']
    .str.contains('Touchscreen', na=False)
    .astype(int)
)
```

[25]:
```
#re-check
laptop.head()
```

[25]:
```
  Company   TypeName  Inches                    ScreenResolution  \
0   Apple  Ultrabook    13.3  IPS Panel Retina Display 2560x1600
1   Apple  Ultrabook    13.3                            1440x900
2      HP   Notebook    15.6                   Full HD 1920x1080
3   Apple  Ultrabook    15.4  IPS Panel Retina Display 2880x1800
4   Apple  Ultrabook    13.3  IPS Panel Retina Display 2560x1600

                          Cpu  Ram               Memory  \
0        Intel Core i5 2.3GHz    8            128GB SSD
1        Intel Core i5 1.8GHz    8  128GB Flash Storage
2  Intel Core i5 7200U 2.5GHz    8            256GB SSD
3        Intel Core i7 2.7GHz   16            512GB SSD
4        Intel Core i5 3.1GHz    8            256GB SSD

                            Gpu  OpSys  Weight  …  Gpu_ARM  Cpu_Intel  \
0  Intel Iris Plus Graphics 640  macOS    1.37  …        0          1
1        Intel HD Graphics 6000  macOS    1.34  …        0          1
2         Intel HD Graphics 620  No OS    1.86  …        0          1
3            AMD Radeon Pro 455  macOS    1.83  …        0          1
4  Intel Iris Plus Graphics 650  macOS    1.37  …        0          1

   Cpu_AMD  Cpu_Samsung  Cpu_GHz  Resolution_Width  Resolution_Height  \
0        0            0      2.3              2560               1600
1        0            0      1.8              1440                900
2        0            0      2.5              1920               1080
3        0            0      2.7              2880               1800
4        0            0      3.1              2560               1600

    Pixels  IPS  Touchscreen
0  4096000    1            0
1  1296000    0            0
2  2073600    0            0
3  5184000    1            0
4  4096000    1            0

[5 rows x 28 columns]
```

[26]:
```
laptop.columns
```

[26]:
```
Index(['Company', 'TypeName', 'Inches', 'ScreenResolution', 'Cpu', 'Ram',
       'Memory', 'Gpu', 'OpSys', 'Weight', 'Price', 'Memory_SSD', 'Memory_HDD',
       'Memory_Hybrid', 'Memory_Flash', 'Gpu_Nvidia', 'Gpu_AMD', 'Gpu_Intel',
       'Gpu_ARM', 'Cpu_Intel', 'Cpu_AMD', 'Cpu_Samsung', 'Cpu_GHz',
       'Resolution_Width', 'Resolution_Height', 'Pixels', 'IPS',
       'Touchscreen'],
      dtype='object')
```

[27]:
```
sub_laptop = laptop[['Company', 'TypeName','OpSys', 'Resolution_Width',␣
↪'Resolution_Height', 'Pixels', 'IPS',\
                     'Touchscreen', 'Inches', 'Weight', 'Ram', 'Memory_SSD',␣
↪'Memory_HDD',\
                      'Memory_Hybrid', 'Memory_Flash', 'Gpu_Nvidia', 'Gpu_AMD',␣
↪'Gpu_Intel', 'Gpu_ARM', 'Cpu_Intel', \
                      'Cpu_AMD', 'Cpu_Samsung', 'Cpu_GHz', 'Price']]
sub_laptop.to_csv('sub_laptop.csv', index=False)
```

[28]:
```
sub_laptop.head()
```

[28]:

	Company	TypeName	OpSys	Resolution_Width	Resolution_Height	Pixels	\
0	Apple	Ultrabook	macOS	2560	1600	4096000	
1	Apple	Ultrabook	macOS	1440	900	1296000	
2	HP	Notebook	No OS	1920	1080	2073600	
3	Apple	Ultrabook	macOS	2880	1800	5184000	
4	Apple	Ultrabook	macOS	2560	1600	4096000	

	IPS	Touchscreen	Inches	Weight	…	Memory_Flash	Gpu_Nvidia	Gpu_AMD	\
0	1	0	13.3	1.37	…	0	0	0	
1	0	0	13.3	1.34	…	128	0	0	
2	0	0	15.6	1.86	…	0	0	0	
3	1	0	15.4	1.83	…	0	0	1	
4	1	0	13.3	1.37	…	0	0	0	

	Gpu_Intel	Gpu_ARM	Cpu_Intel	Cpu_AMD	Cpu_Samsung	Cpu_GHz	Price
0	1	0	1	0	0	2.3	71378.6832
1	1	0	1	0	0	1.8	47895.5232
2	1	0	1	0	0	2.5	30636.0000
3	0	0	1	0	0	2.7	135195.3360
4	1	0	1	0	0	3.1	96095.8080

[5 rows x 24 columns]

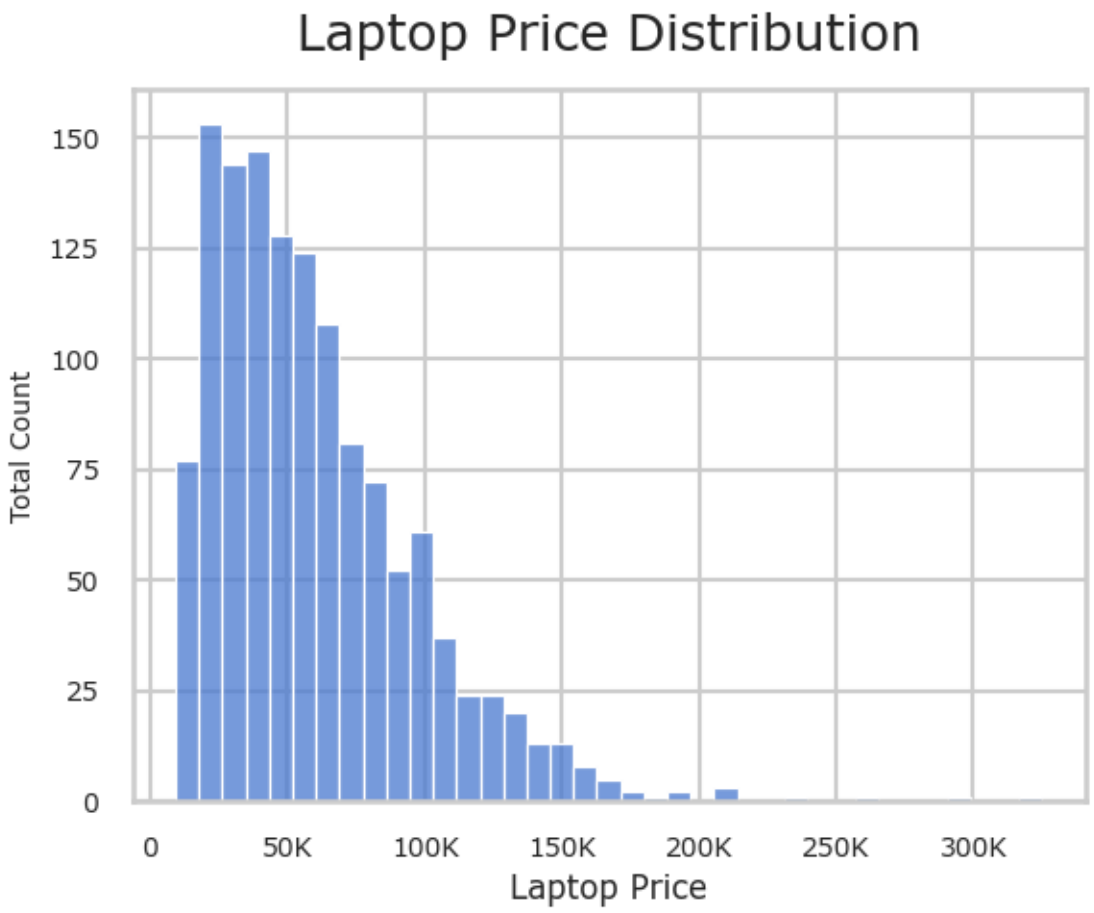
Notes: > TypeName is left as a single because a laptop's core form factor is mutually exclusive. Since laptops categorized strictly as a "Notebook", "Ultrabook", "Gaming", etc.

> Memory is left as multi-label & compositional data because a single laptop can contain multiple independent drives simultaneously, treating this as a single category would have ruined the data. thus it is separated like those.
>
> The Gpu and Cpu Columns is a dimentionality reduction. Since encoding it using the original column data would make the column number astronomically high.

0.2 EDA Phase

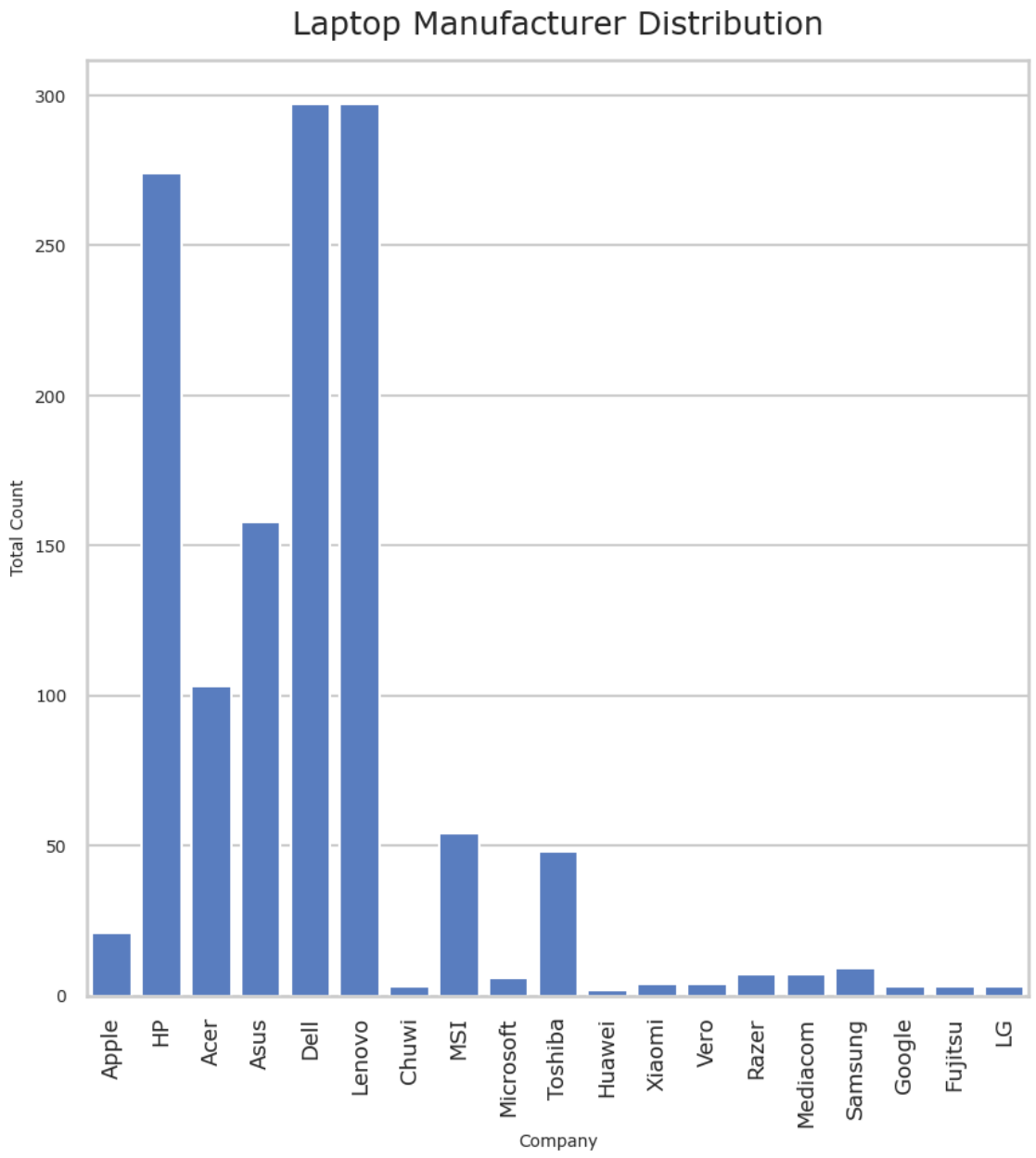
[29]:
```
# Setting overall sns theme for unity

sns.set_theme(
    style="whitegrid",
    context="talk",
    palette="muted",
    font="MS Reference Sans Serif"
)
```

[30]:
```
# Making histogram of price column to check price distribution

sns.histplot(data = sub_laptop, x='Price')
plt.title("Laptop Price Distribution", fontsize=19, pad=15)
plt.xlabel("Laptop Price", fontsize=12)
plt.ylabel("Total Count", fontsize=10)
plt.xticks([0, 50000, 100000, 150000, 200000, 250000, 300000], ['0','50K',␣
 ↪'100K', '150K', '200K', '250K', '300K'], rotation=0, fontsize=10)
plt.yticks(fontsize=10)
plt.savefig('Laptop Price Distribution.png', dpi=300)
plt.show()
```



Apparantly most laptop price sit at around 50000 price range

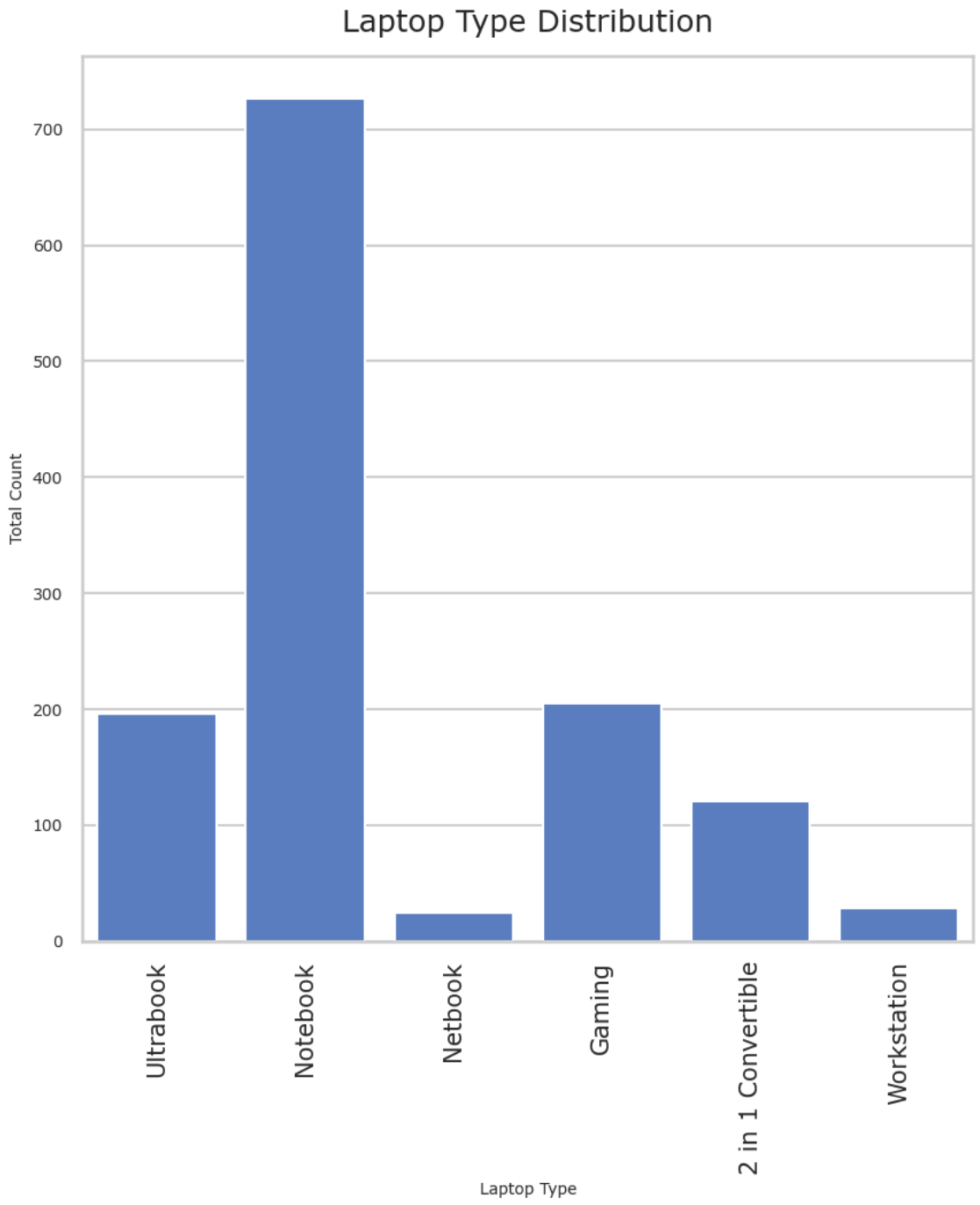
[31]:
```
# Making histogram of Company column to check distribution

plt.figure(figsize=(10, 10))
sns.countplot(data = sub_laptop, x='Company')
plt.title("Laptop Manufacturer Distribution", fontsize=19, pad=15)
plt.xlabel("Company", fontsize=10)
plt.ylabel("Total Count", fontsize=10)
plt.xticks(rotation=90, fontsize=13)
plt.yticks(fontsize=10)
plt.savefig('Laptop Manufacturer Distribution.png', dpi=300)
plt.show()
```



The top 3 laptop company manufacturer Are Dell, Lenovo and HP

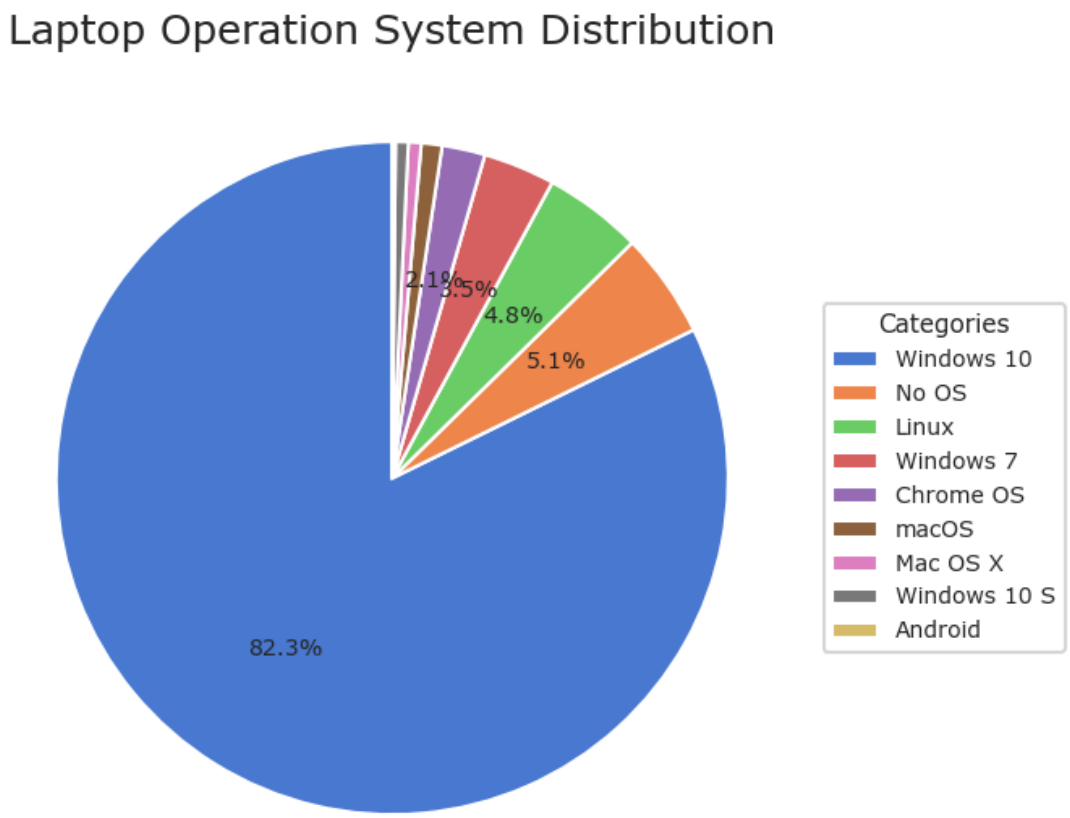
[32]:
```
# Making histogram of TypeName column to check distribution

plt.figure(figsize=(10, 10))
sns.countplot(data = sub_laptop, x='TypeName')
plt.title("Laptop Type Distribution", fontsize=19, pad=15)
plt.xlabel("Laptop Type", fontsize=10)
plt.ylabel("Total Count", fontsize=10)
```

```
plt.xticks(rotation=90, fontsize=15)
plt.yticks(fontsize=10)
plt.savefig('Laptop Type Distribution.png', dpi=300)
plt.show()
```



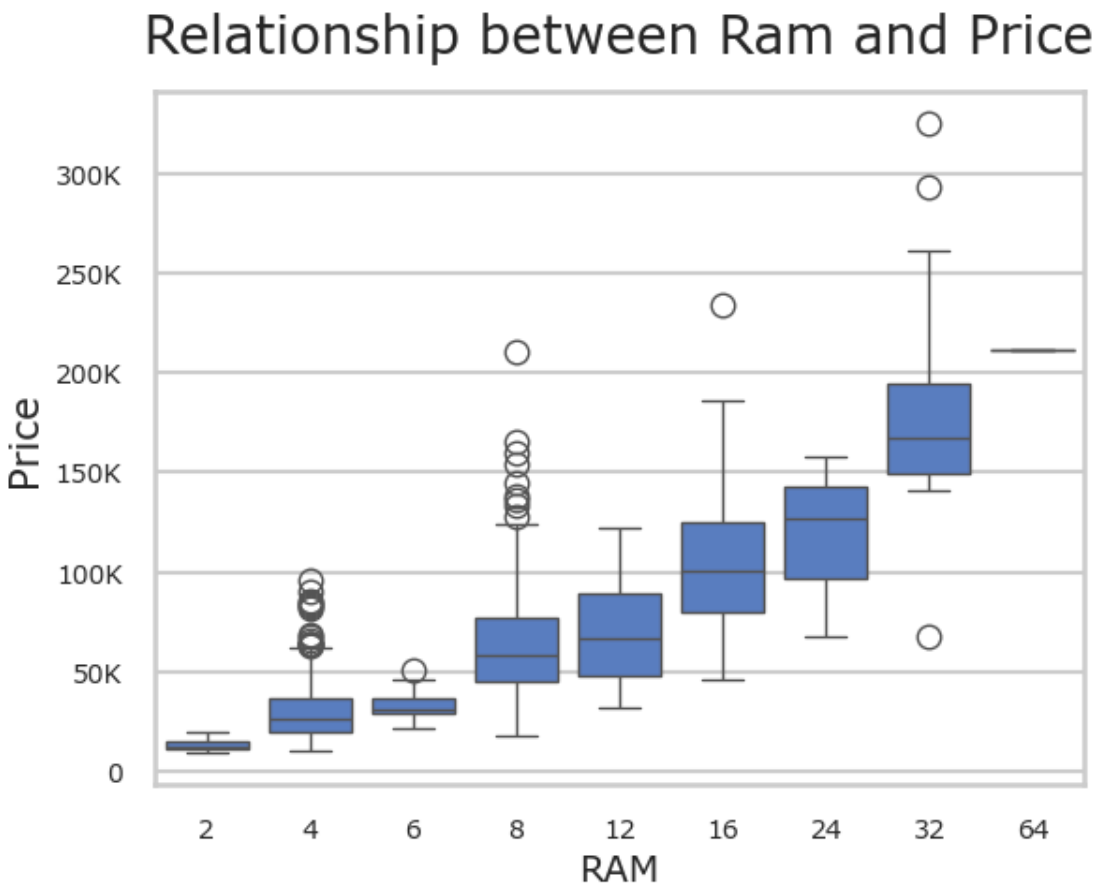
Notebook quantities are more than double the other categories

[33]:
```
# Making pie chart to visualize operation system distribution

# making value counts so it could be visualized properly
class_counts = sub_laptop['OpSys'].value_counts()

labels = class_counts.index
values = class_counts.values

# Find the numerical value of the 5th largest category
# (Since class_counts is already sorted by value_counts, index 4 is the 5th largest)
threshold_value = values[4]

# Create a custom formatting function
def top_5_autopct(pct):
    # Calculate the raw count from the percentage
    total = sum(values)
    val = int(round(pct * total / 100.0))

    # Only return the percentage string if the value is greater than or equal to the 3rd largest
    return f'{pct:.1f}%' if val >= threshold_value else ''

# Plot the pie chart using the custom function
plt.figure(figsize=(8, 6))
plt.pie(
    values,
    autopct=top_5_autopct,     # Pass the custom function here
    startangle=90,
    textprops={'fontsize': 10}
)

# Customizing the legend
plt.legend(labels,
           title="Categories",
           loc="center left",
           bbox_to_anchor=(1, 0.5),
           fontsize=10,
           title_fontsize=11)

plt.title('Laptop Operation System Distribution')
plt.tight_layout()
plt.savefig('Laptop Operation System Distribution.png', dpi=300)
plt.show()
```



Windows 10 is dominating the market by almost 16 times than the category below it.

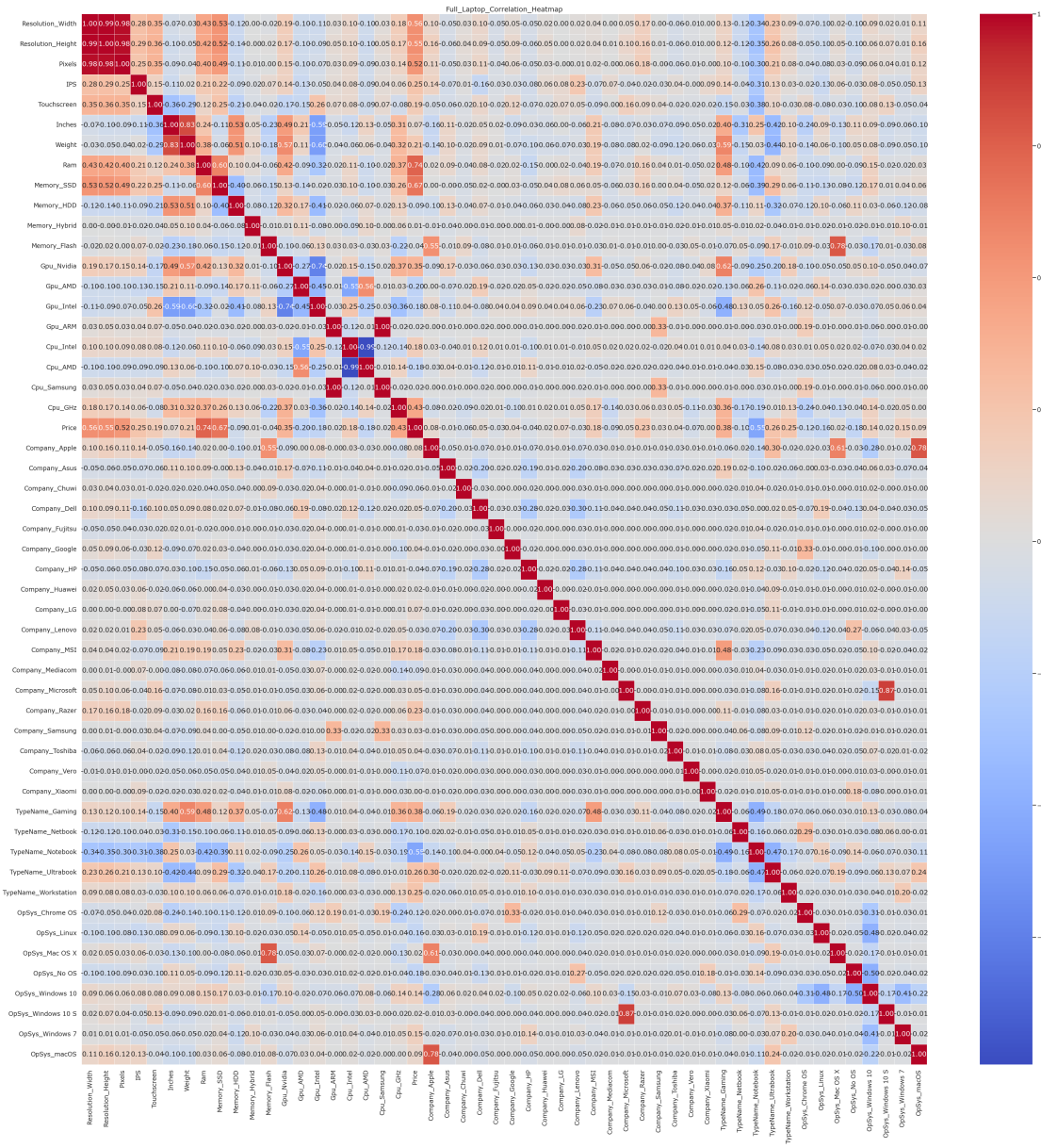
[34]:
```
# Making Boxplot to visualize relationship RAM and price

sns.boxplot(data=sub_laptop, x='Ram', y='Price')
plt.yticks([0, 50000, 100000, 150000, 200000, 250000, 300000], ['0', '50K',␣
↪'100K', '150K', '200K', '250K', '300K'], fontsize=10)
plt.xticks(fontsize=10)
plt.ylabel('Price', fontsize=15)
plt.xlabel('RAM', fontsize=14)
plt.title('Relationship between Ram and Price', fontsize=20, pad=15)
plt.savefig('Relationship between Ram and Price.png', dpi=300)
plt.show()
```



It seems there is a linear relationship between RAM and price, also in the 4GB and 8GB RAM categories contain many outlier with higher price. Another thing to note is there is only 1 outlier with price below the range.

[35]:
```
# Checking correlation between variables using correlation matrix and heatmap

# Encode categorical columns
df_encoded = pd.get_dummies(sub_laptop, drop_first=True)
# Compute correlation matrix
corr_matrix = df_encoded.corr()
# Show correlation matrix
```

[36]:
```
# Create the heatmap
plt.figure(figsize=(50, 50)) # Optional: Adjust figure size
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Full_Laptop_Correlation_Heatmap')
plt.savefig('Full_Laptop_Correlation_Heatmap')
plt.show()
```



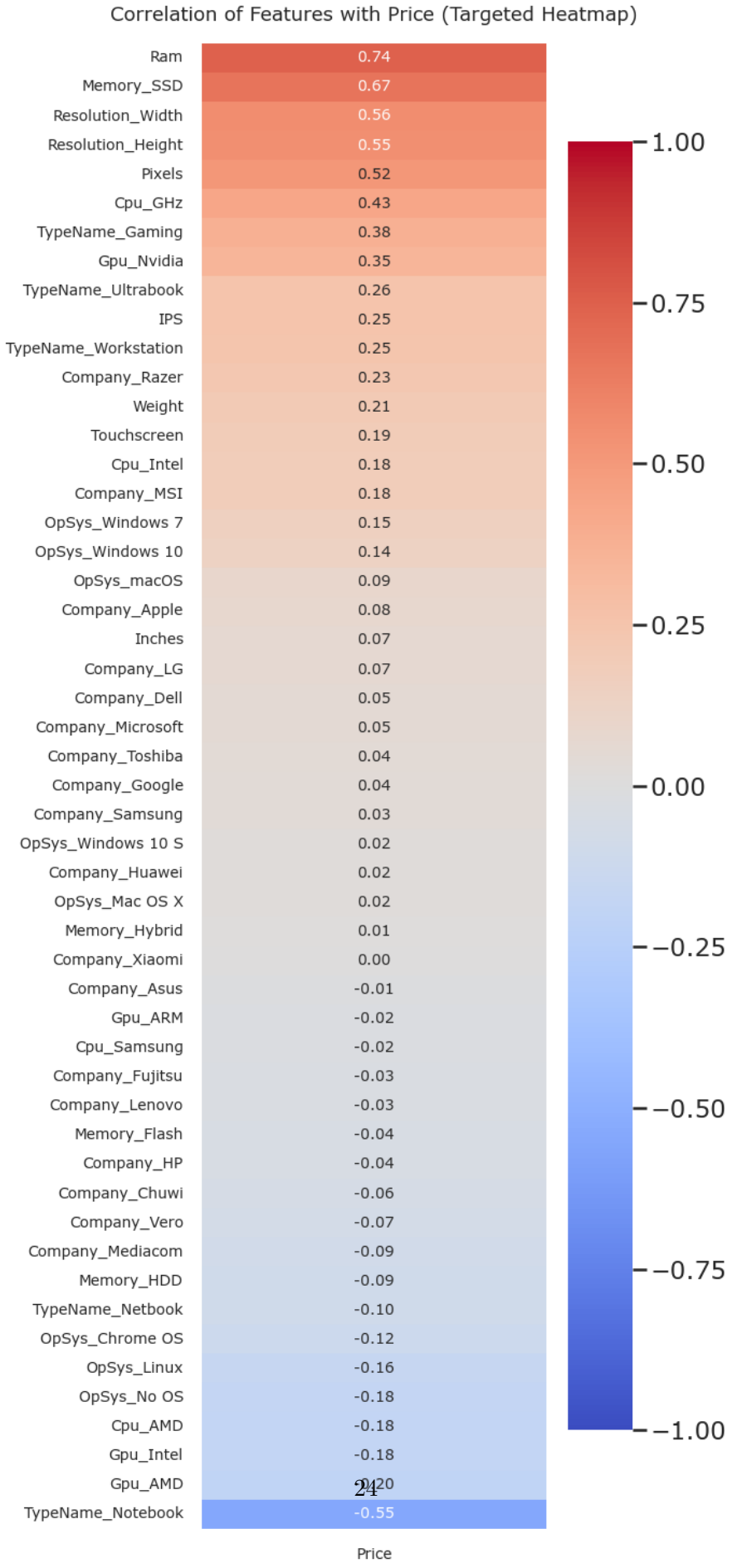
These heatmap is indeed quite unintelligible. But, it is mainly made to check for collinieearity.

Based on the heatmap, variables that has moderate to strong correlation with price are : Resolution_Width (0.56), Resolution_Height (0.55), Pixels (0.52), Memory_SSD (0.74), Memory_HDD (0.67), Typename_notebook (-0.55). Although, variables that correlate with screen are probably in multicollinearity. Thus, its probably better to just include one variable in the modelling to get better result, preferably use pixels. Because pixels could be seen as a summary for the screen width and height column.

There is also a few correlation that is intresting to notes, such as: Gaming laptop is correlated highly with weight, which mean gaming laptop tend to be heavier than other type of laptops. Mac OS X correlate highly with Flash type memory, which mean Mac book tend to use flash memory.

This correlation relate on real life condition, gaming laptop indeed tend to be heavier and flash memory are commonly used by Apple product. There is also some manufacturer and operating system that correlate with each other, like Mac OS correlate with Apple and Windows correlate with Microsoft.

[37]:
```
# Isolate the correlation with the target variable ('Price')
# Drop 'Price' from the index so it doesn't just correlate 1.0 with itself
price_corr = corr_matrix[['Price']].drop('Price').sort_values(by='Price',␣
 ↪ascending=False)


# Making Single-Column Heatmap
plt.figure(figsize=(7, 15))
sns.heatmap(price_corr, annot=True, cmap='coolwarm', fmt=".2f", vmin=-1,␣
 ↪vmax=1, yticklabels=True, annot_kws={"size": 10})
plt.title("Correlation of Features with Price (Targeted Heatmap)", pad=15,␣
 ↪fontsize=13)
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.tight_layout()
plt.savefig('Correlation of Features with Price (Targeted Heatmap).png',␣
 ↪dpi=300)
plt.show()
```


Correlation of Features with Price (Targeted Heatmap)
Ram 0.74
Memory_SSD 0.67
Resolution_Width 0.56
Resolution_Height 0.55
Pixels 0.52
Cpu_GHz 0.43
TypeName_Gaming 0.38
Gpu_Nvidia 0.35
TypeName_Ultrabook 0.26
IPS 0.25
TypeName_Workstation 0.25
Company_Razer 0.23
Weight 0.21
Touchscreen 0.19
Cpu_Intel 0.18
Company_MSI 0.18
OpSys_Windows 7 0.15
OpSys_Windows 10 0.14
OpSys_macOS 0.09
Company_Apple 0.08
Inches 0.07
Company_LG 0.07
Company_Dell 0.05
Company_Microsoft 0.05
Company_Toshiba 0.04
Company_Google 0.04
Company_Samsung 0.03
OpSys_Windows 10 S 0.02
Company_Huawei 0.02
OpSys_Mac OS X 0.02
Memory_Hybrid 0.01
Company_Xiaomi 0.00
Company_Asus -0.01
Gpu_ARM -0.02
Cpu_Samsung -0.02
Company_Fujitsu -0.03
Company_Lenovo -0.03
Memory_Flash -0.04
Company_HP -0.04
Company_Chuwi -0.06
Company_Vero -0.07
Company_Mediacom -0.09
Memory_HDD -0.09
TypeName_Netbook -0.10
OpSys_Chrome OS -0.12
OpSys_Linux -0.16
OpSys_No OS -0.18
Cpu_AMD -0.18
Gpu_Intel -0.18
Gpu_AMD -0.20
TypeName_Notebook -0.55
Price
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

Based on the correlation value to price, I will subset the features that is >0.3 or <-0.3 to filter features that has moderate to strong correlation with Price to make a model with. So, variables that i choose are: Ram, Memory_SSD, Pixels, CPU_GHz, GPU_Nvidia and TypeName.

The reason that I didn't include Resolution_Width and Resolution_Height is because those to variables are already represented by Pixels variables. Thus, if I kept including those two, it will definitely raise a multicollinearity error within the model.

0.3 Advanced Analytics Phase

[38]:
```
# Making variables x & y for modelling using MLR (Multiple Linear Regression)
# This time, x variables are gonna consist of variables that highly correlate␣
 ↪with price. Based on the heat map above.

X1 = pd.get_dummies(
    sub_laptop[['Ram', 'Memory_SSD', 'Pixels', 'Cpu_GHz', 'Gpu_Nvidia',␣
 ↪'TypeName']],
    drop_first=True
)
y = sub_laptop['Price']

#Making model & checking the results summary
X1 = X1.astype(float)
X1 = sm.add_constant(X1)

model1 = sm.OLS(y, X1)
results1 = model1.fit()
results1.summary()
```

[38]:

Dep. Variable:	Price	**R-squared:**	0.741
Model:	OLS	**Adj. R-squared:**	0.739
Method:	Least Squares	**F-statistic:**	369.2
Date:	Sat, 08 Aug 2026	**Prob (F-statistic):**	0.00
Time:	20:50:19	**Log-Likelihood:**	-14683.
No. Observations:	1303	**AIC:**	2.939e+04
Df Residuals:	1292	**BIC:**	2.945e+04
Df Model:	10		
Covariance Type:	nonrobust		

	coef	std err	t	P> |t|	[0.025	0.975]
const	-1963.7258	3188.703	-0.616	0.538	-8219.330	4291.878
Ram	2866.8070	156.589	18.308	0.000	2559.611	3174.003
Memory_SSD	45.6647	4.022	11.352	0.000	37.773	53.556
Pixels	0.0041	0.000	9.082	0.000	0.003	0.005
Cpu_GHz	1.012e+04	1199.636	8.440	0.000	7771.299	1.25e+04
Gpu_Nvidia	266.8923	1578.497	0.169	0.866	-2829.805	3363.590
TypeName_Gaming	5245.0108	2749.310	1.908	0.057	-148.589	1.06e+04
TypeName_Netbook	-4270.2019	4262.487	-1.002	0.317	-1.26e+04	4091.952
TypeName_Notebook	-1.017e+04	1972.749	-5.154	0.000	-1.4e+04	-6296.457
TypeName_Ultrabook	8235.2291	2209.683	3.727	0.000	3900.269	1.26e+04
TypeName_Workstation	4.142e+04	4163.363	9.950	0.000	3.33e+04	4.96e+04

Omnibus:	309.850	**Durbin-Watson:**	2.018
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	1409.882
Skew:	1.047	**Prob(JB):**	7.05e-307
Kurtosis:	7.646	**Cond. No.**	2.58e+07

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.58e+07. This might indicate that there are strong multicollinearity or other numerical problems.

In this model it seems that Gpu_Nvidia and TypeName P-value is higher than alpha threshold. It could be seen that these two variables are insignificant in this model. Maybe because both variables are trying to explain the same variance in this model, thus there might be multicollinearity between these two variables. I will use VIF to check multicollinearity in this model.

[39]:
```
# Checking for multicollinearity using VIF (Variance Inflation Factor)
vif_data = pd.DataFrame()
vif_data["Feature"] = X1.columns
vif_data["VIF"] = [variance_inflation_factor(X1.values, i) for i in␣
 ↪range(len(X1.columns))]
print("--- Variance Inflation Factor ---")
print(vif_data)
print("\n")
```

```
--- Variance Inflation Factor ---
                 Feature        VIF
0                  const  36.563154
1                    Ram   2.277865
2              Memory_SSD   2.060764
3                 Pixels   1.391968
4                 Cpu_GHz   1.325765
5              Gpu_Nvidia   1.906170
6          TypeName_Gaming   3.603546
7         TypeName_Netbook   1.229486
8        TypeName_Notebook   3.451652
9       TypeName_Ultrabook   2.243831
```

```
10  TypeName_Workstation   1.356383
```

The high number in the constant can be ignored. Because it correlates with everything, the numbers are normal to be that high. The interesting part, is that between the two variables in question there is no multicollinearity between them. Or in this case, on any of the variables at all. Since all of the value fall below 5, which translate to low to moderate correlation. Thus it is safe to keep in the model.

In conclusion, even though both variables resulted in a high p-value the overall model is usefull. This is assured by the low Prob (F-statistic) in the model, which act as a whole model p-value. High F-statistic value also proves that the model is usefull overall.

But, since the model show there is a numerical problems. There might be a scaling problems in this model, but the culprit might be int the pixels variable. Since Pixels value are astronomically high compared to other variables, scale problem is a high probability.

[40]:
```
# Making predictor variables from scratch as a preparation for scaling it
X_raw = pd.get_dummies(
    sub_laptop[['Ram', 'Memory_SSD', 'Pixels', 'Cpu_GHz', 'Gpu_Nvidia',␣
 ↪'TypeName']],
    drop_first=True
)

# Convert to float to ensure smooth mathematical scaling
X_raw = X_raw.astype(float)

# Scale the features using StandardScaler
scaler = StandardScaler()
X_scaled_array = scaler.fit_transform(X_raw)

# Put it back into a DataFrame
X_scaled_df = pd.DataFrame(X_scaled_array, columns=X_raw.columns, index=X_raw.
 ↪index)

# Add the constant ONCE at the end
X_scaled_df = sm.add_constant(X_scaled_df)

# Make model & check results summary
model = sm.OLS(y, X_scaled_df)
results = model.fit()
results.summary()
```

[40]:

Dep. Variable:	Price	**R-squared:**	0.741
Model:	OLS	**Adj. R-squared:**	0.739
Method:	Least Squares	**F-statistic:**	369.2
Date:	Sat, 08 Aug 2026	**Prob (F-statistic):**	0.00
Time:	20:50:19	**Log-Likelihood:**	-14683.
No. Observations:	1303	**AIC:**	2.939e+04
Df Residuals:	1292	**BIC:**	2.945e+04
Df Model:	10		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P> |t|**	**[0.025**	**0.975]**
const	5.987e+04	527.342	113.532	0.000	5.88e+04	6.09e+04
Ram	1.457e+04	795.896	18.308	0.000	1.3e+04	1.61e+04
Memory_SSD	8593.9328	757.018	11.352	0.000	7108.813	1.01e+04
Pixels	5650.7677	622.167	9.082	0.000	4430.200	6871.336
Cpu_GHz	5124.6021	607.191	8.440	0.000	3933.413	6315.791
Gpu_Nvidia	123.1021	728.070	0.169	0.866	-1305.227	1551.431
TypeName_Gaming	1909.7656	1001.054	1.908	0.057	-54.103	3873.634
TypeName_Netbook	-585.7866	584.728	-1.002	0.317	-1732.907	561.334
TypeName_Notebook	-5049.0506	979.729	-5.154	0.000	-6971.084	-3127.017
TypeName_Ultrabook	2943.9682	789.928	3.727	0.000	1394.287	4493.650
TypeName_Workstation	6110.8416	614.163	9.950	0.000	4905.976	7315.707

Omnibus:	309.850	**Durbin-Watson:**	2.018
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	1409.882
Skew:	1.047	**Prob(JB):**	7.05e-307
Kurtosis:	7.646	**Cond. No.**	4.96

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Now there is no sign of numerical problems, the Cond. No. also goes do significantly from 2.58e+07 to 4.96.

[41]:
```
# Going to make matemathical equation of the model for presentation

parameters = results.params

# Extract intercept and coefficients
intercept = parameters['const'] if 'const' in parameters else 0.0
coefficients = parameters.drop('const', errors='ignore')

# Build the equation string
equation_terms = [f"({coef:.4f} * {name})" for name, coef in coefficients.
 ↪items()]
equation = "y = " + " + ".join(equation_terms) + f" + ({intercept:.4f})"
print(equation)
```

```
y = (14571.1575 * Ram) + (8593.9328 * Memory_SSD) + (5650.7677 * Pixels) +
(5124.6021 * Cpu_GHz) + (123.1021 * Gpu_Nvidia) + (1909.7656 * TypeName_Gaming)
```

```
+ (-585.7866 * TypeName_Netbook) + (-5049.0506 * TypeName_Notebook) + (2943.9682
* TypeName_Ultrabook) + (6110.8416 * TypeName_Workstation) + (59870.0429)
```

I will also save the summary statistics results by turning it into a saveable figure.

[42]:
```
# Extract the coefficients and statistics from your fitted model into a␣
 ↪DataFrame
results_df = pd.DataFrame({
    'Coefficient': results.params.round(2),
    'Std. Error': results.bse.round(2),
    't-value': results.tvalues.round(2),
    'P > |t|': results.pvalues.round(4),
})

# Set up the matplotlib figure
fig, ax = plt.subplots(figsize=(8, 4))
ax.axis('off')
ax.axis('tight')

# Create a clean table plot
table = ax.table(
    cellText=results_df.values,
    colLabels=results_df.columns,
    rowLabels=results_df.index,
    loc='center',
    cellLoc='center'
)

# Style the table for a professional presentation look
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1.2, 1.2)

# Save the table as a high-resolution image
plt.title("Multiple Linear Regression Results (Scaled)", fontsize=14,␣
 ↪fontweight='bold', pad=20)
plt.savefig("Multiple Linear Regression Results (Scaled).png",␣
 ↪bbox_inches='tight', dpi=300)
plt.show()
```

Multiple Linear Regression Results (Scaled)

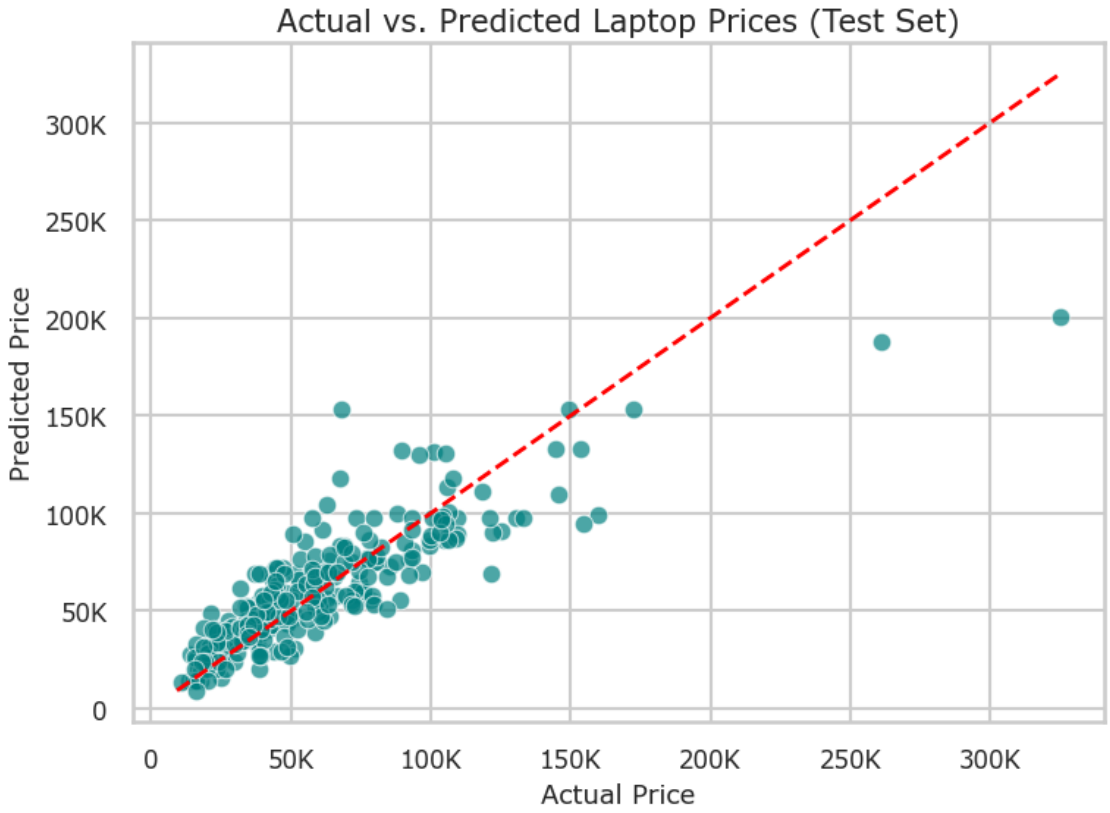
	Coefficient	Std. Error	t-value	P > |t|
const	59870.04	527.34	113.53	0.0
Ram	14571.16	795.9	18.31	0.0
Memory_SSD	8593.93	757.02	11.35	0.0
Pixels	5650.77	622.17	9.08	0.0
Cpu_GHz	5124.6	607.19	8.44	0.0
Gpu_Nvidia	123.1	728.07	0.17	0.8658
TypeName_Gaming	1909.77	1001.05	1.91	0.0566
TypeName_Netbook	-585.79	584.73	-1.0	0.3166
TypeName_Notebook	-5049.05	979.73	-5.15	0.0
TypeName_Ultrabook	2943.97	789.93	3.73	0.0002
TypeName_Workstation	6110.84	614.16	9.95	0.0

Since the model are already proven to be quite sufficient. It is time to test the model using Train-Test Split

[43]:
```
# Performing Train-Test Split (using 80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X_scaled_df, y, test_size=0.
 ↪2, random_state=42)

# Add constant for statsmodels
X_train_const = sm.add_constant(X_train)
X_test_const = sm.add_constant(X_test)

# Train the model on the TRAINING data
model = sm.OLS(y_train, X_train_const).fit()

# Make predictions on the TESTING data
y_pred = model.predict(X_test_const)

# Calculate Error Metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"Mean Absolute Error (MAE): {mae:,.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:,.2f}")

# 7. Visualize Actual vs Predicted
plt.figure(figsize=(8, 6))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.7, color='teal')
plt.plot([y.min(), y.max()], [y.min(), y.max()], color='red', linestyle='--',␣
 ↪lw=2) # Perfect prediction line
plt.title("Actual vs. Predicted Laptop Prices (Test Set)", fontsize=15)
plt.xlabel("Actual Price", fontsize=13)
plt.ylabel("Predicted Price", fontsize=13)
```

```
plt.xticks([0, 50000, 100000, 150000, 200000, 250000, 300000], ['0','50K',␣
  ↪'100K', '150K', '200K', '250K', '300K'], fontsize=12)
plt.yticks([0, 50000, 100000, 150000, 200000, 250000, 300000], ['0','50K',␣
  ↪'100K', '150K', '200K', '250K', '300K'], fontsize=12)
plt.tight_layout()
plt.savefig('Actual vs. Predicted Laptop Prices (Test Set).png', dpi=300)
plt.show()
```

```
Mean Absolute Error (MAE): 13,638.00
Root Mean Squared Error (RMSE): 19,419.81
```



The train-test split evaluation confirms that the model generalizes well to unseen data, showing strong predictive accuracy for budget and mid-tier laptops along the identity line. While the model under-predicts extreme high-end luxury price points due to data scarcity at the upper tail, the overall fit demonstrates a reliable baseline for market pricing.

Also according to the MAE and RMSE, on average the model's predictions deviate by an MAE of 13,638.00, while the RMSE of 19,419.81 highlights occasional larger errors driven by extreme-priced outliers.